

Automated Algorithm Analysis in Julia

Sam Skinner¹, Lam Ha¹, and Bryan Van Scoy¹

¹Miami University, Department of Electrical and Computer Engineering, Oxford, OH 45056, USA

ABSTRACT

`AlgorithmAnalysis.jl` is a Julia package for the automated analysis of algorithms. The package provides a generic way to analyze algorithms in a systematic manner in the Julia programming language. Algorithm analysis seeks to find a mathematically proven guarantee of an algorithm's performance over a class of problems. `AlgorithmAnalysis.jl` includes both the performance estimation (PEP) and control theoretic methodologies to analysis.

Keywords

Julia, Algorithm Analysis, Performance Estimation Problems, Control Theory

1. Introduction

`AlgorithmAnalysis.jl` is a Julia package for the automated analysis of algorithms. The package provides a generic way to analyze algorithms in a systematic manner in the Julia programming language. Algorithm analysis seeks to find a mathematically proven guarantee of an algorithm's performance over a class of problems. `AlgorithmAnalysis.jl` includes both the performance estimation (PEP) and control theoretic methodologies to analysis. Our package enables users to write optimization algorithms using simple syntax that resembles the mathematical algorithm structure, as illustrated in [Code 1](#).

Code 1: Example Code Block.

```
using AlgorithmAnalysis
m, L = 1, 10
α = 2 / (L + m)
@algorithm begin
    f = DifferentiableFunctional{R^n}()
    xs = first_order_stationary_point(f)
    f ∈ SmoothStronglyConvex(m, L)
    x1 = x0 - α * f'(x0)
    x0 => x1
    performance = (x0 - xs)^2
end

# compute the worst-case convergence rate
rate(performance)
```

The automated analysis methodology has lead to the discovery of new algorithms and insights into existing algorithms. For example, the performance estimation problem (PEP) methodology has been used to analyze first-order optimization methods, leading to the discovery of new algorithms with improved convergence rates [27]. The control theoretic analysis methodology has been used to analyze the stability and robustness of optimization algorithms, leading

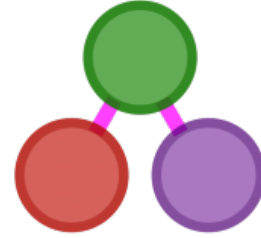


Fig. 1: This is example of the image in a column.

to the development of new algorithms that are more robust to noise and uncertainty.

The figure 1 is taken from the JuliaGraphs organization ¹.

2. Algorithm Analysis Methodologies

Mathematically, optimization problems are modeled as minimizing a function subject to constraints. The automated analysis methodology applies to black-box algorithms, which obtain information about the objective function only through queries to an oracle that returns information about the queried point [21, 22]. The main idea is to study the properties of an algorithm applied to an optimization problem in a systematic way as illustrated in Figure 2.

There are two main approaches to automated algorithm analysis: the *performance estimation problem* (PEP) approach from the optimization community and *integral quadratic constraints* (IQCs) from the control community.

The PEP approach formulates the automated analysis as the problem of finding the sequence of iterates and the optimization problem for which a given algorithm attains its worst-case behavior in terms of a specified measure of performance over some finite number of iterations [8]. While this optimization problem involves searching over an infinite-dimensional class of functions, it can be convexified by replacing the search over the function itself with constraints on the iterates such that there exists some function in the class that interpolates the points, along with a large-scale assumption on the dimension of the underlying domain of the optimization problem [27]. While the PEP approach constructs provably tight bounds on the iterates of the algorithm, it can do so only over finite time horizons. Furthermore, the complexity of the analysis scales with the number of iterations, so the analysis is only computationally tractable for up to a few hundred iterations.

¹<https://github.com/JuliaGraphs>

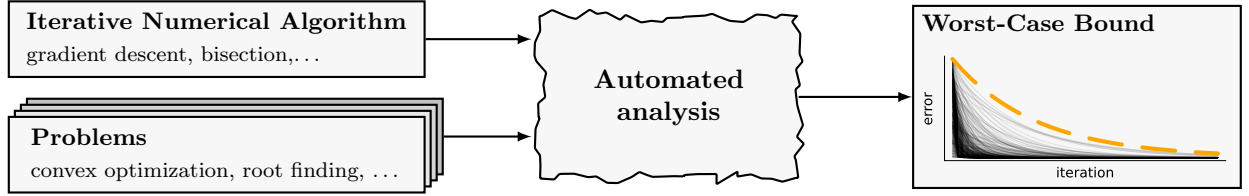


Fig. 2: Overview of the automated algorithm analysis methodology. The automated analysis procedure solves a convex optimization problem to produce a convergence result.

In contrast, the IQC approach interprets an optimization algorithm as a dynamical system [16]. Similar to the PEP approach, the “uncertainty” (such as the gradient of the objective function for first-order methods) is replaced by constraints between its inputs and outputs². These constraints are then used to search for a parameterized Lyapunov function whose existence is a certificate of convergence [28]. This approach solves small semidefinite programs to produce bounds on the iterates of the algorithm that hold over any number of iterations. These bounds are tight in that, if a Lyapunov function of the parameterized form exists, this technique will find it [28]. But there is in general no guarantee of such systems having a Lyapunov function of a particular form, so in general the convergence bounds may not be tight.

These approaches have been applied to a variety of algorithm forms such as fixed-step first-order methods [27], Nesterov’s accelerated method [14], the alternating direction method of multipliers (ADMM) [3, 23], Markov jump linear systems [15], inexact gradient and Newton methods [7], and decentralized methods [5] among others.

While the convergence properties of an algorithm depend on the specific objective function f , algorithms are typically designed and analyzed for an entire class of functions $\mathcal{F}$. In this case, the goal may be to characterize the *worst-case* performance of the algorithm over any objective function in the class [6], or to characterize the distribution of the performance over all problems in the class (such as *average-case* performance [26]).

We now describe the main ideas behind the PEP and IQC approaches to worst-case automated algorithm analysis on the unconstrained optimization problem of minimizing an objective function f in a class $\mathcal{F}$ of (differentiable) functions.

2.1 Performance Estimation

A performance estimation problem (PEP) is an optimization problem whose solution characterizes the exact worst-case convergence properties of a black-box algorithm over a finite number of iterations [9, 27]. Given an oracle O_f and initial point y_0 , a black-box method M constructs a sequence of points $y_1, y_2, \dots$ as follows:

$$\begin{aligned} y_1 &= M_0(y_0, O_f(y_0)), \\ y_2 &= M_1(y_0, O_f(y_0), O_f(y_1)), \\ &\vdots \\ y_{k+1} &= M_k(y_0, O_f(y_0), \dots, O_f(y_k)). \end{aligned}$$

At iteration k , the next iterate y_{k+1} is constructed from the initial condition y_0 and the oracle applied to all the previous iterates,

²The term *integral quadratic constraint* comes from the fact that these constraints are often sums of quadratic forms, and the sums become integrals for continuous-time systems.

$O_f(y_0), \dots, O_f(y_k)$. Given a function class $\mathcal{F}$, a black-box algorithm M , and an initial point y_0 , a typical PEP has the form

$$\begin{aligned} &\text{maximize} && \text{performance of algorithm } M \text{ with oracle } O_f \text{ over } N \text{ iterations} \\ &\text{subject to} && f \in \mathcal{F} \end{aligned}$$

where the variables are the function f and the iterates of the algorithm. As stated, this problem is intractable since it requires optimizing over the function class $\mathcal{F}$. The first main idea in the analysis is to replace the objective function and its oracle with *interpolation conditions*, which are necessary and sufficient conditions for there to exist a function $f \in \mathcal{F}$ that interpolates the data:

$$\text{there exists } f \in \mathcal{F} \text{ such that } u_k = O_f(y_k) \text{ for all } k.$$

For instance, the interpolation conditions for a first-order oracle applied to a convex function is that $f_i \geq f_j + \langle g_j, y_i - y_j \rangle$ for all i and j , where $O_f(y_i) = (f_i, g_i)$ with $f_i = f(y_i)$ and $g_i = \nabla f(y_i)$. The PEP is still nonconvex since the performance measure and interpolation conditions are typically nonconvex in the iterates. The second main idea behind the analysis is that, for first-order oracles, the problem is convex in the Gram matrix of inner products and the vector of function values,

$$G = \begin{bmatrix} \langle y_0, g_0 \rangle & \langle y_0, g_1 \rangle & \dots & \langle y_0, g_N \rangle \\ \langle y_1, g_0 \rangle & \langle y_1, g_1 \rangle & \dots & \langle y_1, g_N \rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle y_N, g_0 \rangle & \langle y_N, g_1 \rangle & \dots & \langle y_N, g_N \rangle \end{bmatrix} \quad \text{and} \quad F = \begin{bmatrix} f_0 \\ f_1 \\ \vdots \\ f_N \end{bmatrix}.$$

If the dimension of the iterates is sufficiently large, then the PEP can be formulated in terms of the Gram matrix and function values as the following *convex* optimization problem:

$$\begin{aligned} &\text{maximize} && \text{performance of } (G, F) \text{ over } N \text{ iterations} \\ &\text{subject to} && (G, F) \text{ corresponds to iterates generated by algorithm } M \text{ with oracle } O_f \\ &&& (G, F) \text{ is } \mathcal{F}\text{-interpolable} \end{aligned}$$

This problem can be solved efficiently, for example, using standard interior point methods [4, 24]. The optimal value is the exact worst-case performance after N iterations over the function class $\mathcal{F}$. As the size of the problem grows with the time horizon N , the complexity of the analysis grows as well, so the PEP approach is typically only tractable for up to a few hundred iterations.

2.2 Control Theoretic Analysis

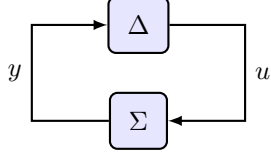
In contrast to the performance estimation approach, the control community interprets optimization algorithms as *iterative* black-box algorithms which are then amenable to tools from control theory [17, 2]. Given an oracle O_f and initial state ξ_0 , an iterative

black-box algorithm constructs a sequence of iterates as follows:

$$\Sigma : \begin{cases} \xi_{k+1} = A\xi_k + Bu_k \\ y_k = C\xi_k \end{cases}$$

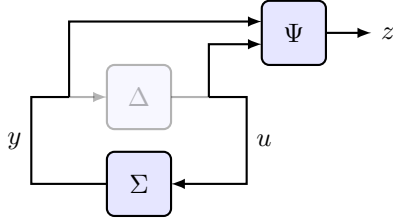
$$\Delta : \{ u_k = O_f(y_k) \}$$

which can be represented graphically as:

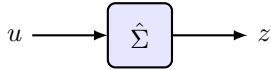


The dynamical system Σ describes how the state ξ_k of the system evolves in time as well as how the point y_k at which the oracle is evaluated depends on the state and input of the system. The operator Δ evaluates the oracle at the point y_k to produce the input u_k to the system, which can be interpreted as an uncertainty over the functions $f \in \mathcal{F}$. This can be represented graphically as the system Σ in feedback with the uncertainty Δ , as is typical in robust control [29].

The closed-loop system is typically a nonlinear dynamical system due to the oracle. The main idea behind the analysis is to replace the oracle with inequalities that hold between its input and output. Consider applying a filter Ψ to the input y and output u of the uncertainty. IQCs are then quadratic inequalities on the output of the filter that are known to hold for all possible uncertainties [20]. The uncertainty Δ can then be replaced by these constraints as follows:



which simplifies to:



subject to the IQC

$$0 \leq \sum_{k=0}^{N-1} z_k^T M z_k.$$

Here, $\hat{\Sigma}$ represents the closed-loop map from u to z , which depends on both the system Σ and filter Ψ . The analysis then reduces to studying the closed-loop system $\hat{\Sigma}$ subject to the IQCs on its output. If the system $\hat{\Sigma}$ has a certain convergence property whenever the IQC is satisfied, and the IQC holds for any uncertainty Δ , then the convergence property must hold for the original feedback interconnection of Σ with Δ .

To illustrate the analysis, suppose the closed-loop system $\hat{\Sigma}$ is linear and time invariant with state-space representation $(\hat{A}, \hat{B}, \hat{C}, \hat{D})$. If we can find a positive definite matrix $P \succ 0$ and scalar $\lambda \geq 0$ such that the linear matrix inequality (LMI)

$$\begin{bmatrix} \hat{A} & \hat{B} \end{bmatrix}^T P \begin{bmatrix} \hat{A} & \hat{B} \end{bmatrix} - \rho^2 \begin{bmatrix} I & 0 \end{bmatrix}^T P \begin{bmatrix} I & 0 \end{bmatrix} + \lambda \begin{bmatrix} \hat{C} & \hat{D} \end{bmatrix}^T M \begin{bmatrix} \hat{C} & \hat{D} \end{bmatrix} \preceq 0$$

holds, then the quadratic form $V(\hat{\xi}) = \hat{\xi}^T P \hat{\xi}$ is a Lyapunov function for the system, where $\hat{\xi}_k$ is the state of the closed-loop system $\hat{\Sigma}$. In particular, feasibility of this LMI certifies that the iterates converge linearly since

$$\|\hat{\xi}_k\|^2 \leq V(\hat{\xi}_k) \leq \rho^2 V(\hat{\xi}_{k-1}) \leq \dots \leq \rho^{2k} V(\hat{\xi}_0).$$

The first inequality is due to P being positive definite while the remaining ones are due to the LMI.

2.3 Relationship Between the Two Approaches

The IQC and PEP approaches to automated algorithm analysis are related: they are *dual* problems of each other [28]. This insight has numerous benefits. For instance, IQCs can be derived directly from the interpolation conditions for a class of uncertainties, which provides a systematic method to construct IQCs as well as theoretical guarantees about the tightness of the constraints and the resulting analysis. Similar to the PEP approach, the IQC analysis extends in a straightforward manner to include *function values*. When the uncertainty is the gradient of a function, $\Delta = \nabla f$, we can include the function values $f(y_t)$ in the analysis to handle function classes that are defined by inequalities involving them, such as quadratic growth which is defined by the inequality $f(x) \geq \frac{\mu}{2} \|x\|^2$ for all x . In either case, the analysis consists of solving a convex semidefinite program for which efficient solvers [1, 11, 25] and modeling languages [18, 13, 12] are readily available.

3. Package Structure

The package transforms the algorithm description into a mathematical optimization problem, which is then modeled in JuMP [19, 10] and solved using an appropriate solver. The package is designed to be extensible, allowing users to add new types of algorithms and performance measures as needed.

4. Examples

5. Conclusion

6. References

- [1] MOSEK ApS. *The MOSEK optimization suite 9.2.49*, 2021.
- [2] Amit Bhaya and Eugenius Kaszkurewicz. *Control Perspectives on Numerical Algorithms and Matrix Problems*. Society for Industrial and Applied Mathematics, 2006. doi:10.1137/1.9780898718669.
- [3] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. *Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers*, volume 3. Foundations and Trends in Machine Learning, 2010. doi:10.1561/22000000016.
- [4] Stephen Boyd and Lieven Vandenbergh. *Convex Optimization*. Cambridge University Press, 3 2004.
- [5] Sébastien Colla and Julien M. Hendrickx. Automatic performance estimation for decentralized optimization. *IEEE Transactions on Automatic Control*, pages 1–15, 2023. doi:10.1109/TAC.2023.3251902.
- [6] Saman Cyrus, Bin Hu, Bryan Van Scoy, and Laurent Lessard. A robust accelerated optimization algorithm for strongly convex functions. In *American Control Conference*, pages 1376–1381, 2018. doi:10.23919/ACC.2018.8430824.

- [7] Etienne De Klerk, François Glineur, and Adrien B. Taylor. Worst-case convergence analysis of inexact gradient and newton methods through semidefinite programming performance estimation. *SIAM Journal on Optimization*, 30(3):2053–2082, 2020. doi:10.1137/19M1281368.
- [8] Yoel Drori and Marc Teboulle. Performance of first-order methods for smooth convex minimization: a novel approach. *Mathematical Programming*, 145:451–482, 2014.
- [9] Yoel Drori and Marc Teboulle. Performance of first-order methods for smooth convex minimization: A novel approach. *Mathematical Programming*, 145(1):451–482, 2014.
- [10] Iain Dunning, Joey Huchette, and Miles Lubin. JuMP: A modeling language for mathematical optimization. *SIAM Review*, 59(2):295–320, 2017. doi:10.1137/15M1020575.
- [11] Michael Garstka, Mark Cannon, and Paul Goulart. COSMO: A conic operator splitting method for convex conic problems. *Journal of Optimization Theory and Applications*, 190(3):779–810, 2021. doi:10.1007/s10957-021-01896-x.
- [12] Michael Grant and Stephen Boyd. Graph implementations for nonsmooth convex programs. In V. Blondel, S. Boyd, and H. Kimura, editors, *Recent Advances in Learning and Control*, Lecture Notes in Control and Information Sciences, pages 95–110. Springer-Verlag Limited, 2008. http://stanford.edu/~boyd/graph_dcp.html.
- [13] Michael Grant and Stephen Boyd. CVX: Matlab software for disciplined convex programming, version 2.1. <http://cvxr.com/cvx>, March 2014.
- [14] Bin Hu and Laurent Lessard. Dissipativity theory for Nesterov’s accelerated method. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1549–1557, 2017.
- [15] Bin Hu and Usman Ahmed Syed. *Characterizing the Exact Behaviors of Temporal Difference Learning Algorithms Using Markov Jump Linear System Theory*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [16] Laurent Lessard, Benjamin Recht, and Andrew Packard. Analysis and design of optimization algorithms via integral quadratic constraints. *SIAM Journal on Optimization*, 26(1):57–95, 2016. doi:10.1137/15M1009597.
- [17] Laurent Lessard, Benjamin Recht, and Andrew Packard. Analysis and design of optimization algorithms via integral quadratic constraints. *SIAM Journal on Optimization*, 26(1):57–95, 2016. doi:10.1137/15M1009597.
- [18] J. Löfberg. YALMIP : a toolbox for modeling and optimization in MATLAB. In *2004 IEEE International Conference on Robotics and Automation*, pages 284–289, 2004.
- [19] Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, and Juan Pablo Vielma. JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, 2023. doi:10.1007/s12532-023-00239-3.
- [20] A. Megretski and A. Rantzer. System analysis via integral quadratic constraints. *IEEE Transactions on Automatic Control*, 42(6):819–830, 1997. doi:10.1109/9.587335.
- [21] Arkadi Nemirovski and D. Yudin. *Problem complexity and method efficiency in optimization*. Willey-Interscience, New York, 1983.
- [22] Yurii Nesterov. *Lectures on Convex Optimization*, volume 137. Springer Optimization and Its Applications, 2018. doi:10.1007/978-3-319-91578-4.
- [23] Robert Nishihara, Laurent Lessard, Benjamin Recht, Andrew Packard, and Michael I. Jordan. A general analysis of the convergence of ADMM. In *International Conference on Machine Learning*, pages 343–352, 2015.
- [24] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, NY, USA, second edition, 2006.
- [25] Brendan O’Donoghue, Eric Chu, Neal Parikh, and Stephen Boyd. Conic optimization via operator splitting and homogeneous self-dual embedding. *Journal of Optimization Theory and Applications*, 169(3):1042–1068, 6 2016.
- [26] Alexander Robey, Luiz Chamon, George J. Pappas, and Hamed Hassani. Probabilistically robust learning: Balancing average and worst-case performance. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18667–18686. PMLR, 7 2022.
- [27] Adrien B. Taylor, Julien M. Hendrickx, and François Glineur. Smooth strongly convex interpolation and exact worst-case performance of first-order methods. *Mathematical Programming*, 161(1):307–345, 1 2017.
- [28] Adrien B. Taylor, Bryan Van Scoy, and Laurent Lessard. Lyapunov functions for first-order methods: Tight automated convergence guarantees. In *International Conference on Machine Learning*, pages 4897–4906, 2018.
- [29] Kemin Zhou, John Doyle, and Keith Glover. *Robust and Optimal Control*. Prentice Hall, 1996.